

Megaprompt — GENARO · Catálogo

Pegar entero en Claude Code, parado en `~/Desktop/UAI/MDW-project/ecommerce-mdw`.

Sos el desarrollador a cargo del **catálogo** del e-commerce de BC Importados, el trabajo final de Metodologías y Desarrollo Web (UAI), y además sos el **responsable del repositorio** (Unknowns24): sos el único que mergea a `main`. Trabajás con otras tres personas: Yasmín (identidad y RBAC), Agustín (kernel, infraestructura, inventario y pagos) y Nicolás (carrito, checkout y pedidos).

Hoy es domingo 04/10/2026. El Parcial I es el martes 06/10 y se defiende el backend corriendo en producción. Si la URL pública no responde o el login no funciona, el parcial no se toma y pasa a la recuperación del 24/11.

Tu módulo carga dos criterios pesados de la rúbrica:

- **Modelo de datos, 30%.** El núcleo obligatorio pide 5+ *entidades propias del dominio, con una relación 1-N y una N-N, y CRUD completo sobre al menos una*. El **CRUD completo es el de Producto**, y es tuyo.
- **API, 25%.** El catálogo público es la mitad de los endpoints del sistema.

Estado real del repositorio

- Ya existe `backend/config/viper.go`, que es el **único** lugar del backend donde se leen variables de entorno (ADR-001).
- Agustín está subiendo el **kernel**: `cmd/api/main.go`, la conexión a Postgres, el migrador y el paquete `apierr`. Esperá a que esté mergeado en `develop` antes de escribir handlers; el dominio y la migración podés empezarlos ya.
- Yasmín está subiendo el **middleware de sesión y permisos**. Tus endpoints de administración lo necesitan; los públicos no.
- `docs/specs.md` tiene tus historias: `SPEC-H04` (catálogo, búsqueda y orden), `SPEC-H05` (detalle de variante) y `SPEC-H08` (gestión de catálogo).
- `docs/erd.dbml` tiene `marca`, `categoria`, `categoria_jerarquia`, `producto`, `producto_categoria`, `variante`, `imagen_variante`.

El modelo, en una frase

Un **Producto** es una línea (por ejemplo "Crema Victoria's Secret Love Spell"); una **Variante** es lo que realmente se vende (el frasco de 236 ml, con su código y su precio). El catálogo público muestra **variantes**, no productos. El stock, los pedidos y el carrito apuntan a **variante**.

Decisiones ya tomadas que tenés que respetar

- Go 1.26 · Chi · GORM · gormigrate · PostgreSQL (ADR-001, 002 y 003).
- **Plata en int64 de centavos**, columnas `bigint`, campos `..._centavos`. Nunca `float64`: no representa exacto 0,10 y los errores se acumulan al sumar. Es el clásico que aparece en la defensa.
- **Identificadores uuid.UUID**, columnas `uuid`. Nunca autoincremental.
- Nombres de tablas y columnas en castellano y `snake_case`, como el ERD.

- Capas: el handler traduce HTTP y nada más; las reglas viven en `usecases` ; las consultas en `repositories` . Un handler no arma un `db.Where(...)` .

Reglas duras

1. **Sólo tocás los archivos que esta consigna te asigna.** No edites archivos de Yasmín, Agustín ni Nicolás. `go.mod` , `main.go` , `router.go` y `registry.go` son de Agustín: si te falta una dependencia, pedísela por el grupo.
2. `go build ./...` y `go vet ./...` en verde antes de cada push.
3. **Nunca `db.Create(&body)` .** El body pasa por un DTO validado, y de ahí se copian campo por campo los que corresponden. Si pasás el body directo, cualquier columna que exista pasa a ser escribible desde afuera.
4. **Todo listado lleva `Limit` y `Select` explícitos.** Sin límite, el día que haya 40.000 variantes el endpoint las trae todas.
5. Cada commit en español, en imperativo, y es una pieza que compila. Al terminar cada bloque: `git add` , `git commit` , `git push` . **No acumules trabajo sin pushear:** la nota es individual y se mide sobre el historial del repositorio.
6. Rama: `feat/catalogo` , salida de `develop` . `main` está protegida.

```
git fetch --all
git checkout develop && git pull
git checkout -b feat/catalogo
```

BLOQUE 0 — Lo que te toca como responsable del repositorio (15 minutos, hacelo primero)

1. En GitHub → Settings → Branches, verificá que `main` esté protegida: nadie pushea directo y nadie aprueba su propio Pull Request.
2. Creá `.github/CODEOWNERS` para que cada PR le pida review a quien corresponde:

```
/backend/internal/core/domain/identidad.go    @yasmin
/backend/internal/adapters/rest/middleware/    @yasmin
/backend/internal/adapters/jwt/                @yasmin
/backend/internal/core/domain/catalogo.go       @genaro
/backend/internal/core/domain/inventario.go     @agustin
/backend/cmd/                                  @agustin
/backend/go.mod                                @agustin
/backend/internal/core/domain/pedidos.go        @nicolas
/docs/specs.md                                @yasmin @genaro @agustin @nicolas
```

(Poné los usuarios reales de GitHub de cada uno.)

1. `.github/pull_request_template.md` con el checklist de code review de la cátedra:

- [] ¿Algún endpoint quedó sin middleware de sesión y sin decir que es público a propósito?
- [] ¿Alguna consulta devuelve datos de un usuario sin su id en el WHERE?
- [] ¿Aparece `rol` o `usuarioID` en un DTO, un header propio o la query?
- [] ¿Algún listado sin Limit o sin Select?
- [] ¿Alguna regla de negocio quedó adentro de un handler?
- [] ¿Las rutas coinciden con docs/api/?
- [] ¿docs/api/*.http tiene los tres casos de cada endpoint nuevo?
- [] ¿go build, go vet y go test en verde?

```
git add .github/
git commit -m "chore(repo): CODEOWNERS y plantilla de PR con el checklist de code review"
git push -u origin feat/catalogo
```

BLOQUE 1 — Dominio y migración

1.1 Entidades — backend/internal/core/domain/catalogo.go

Marca	id, nombre (único), estado (ACTIVA INACTIVA), creado_en
Categoria	id, nombre, padre_id (nullable, autorreferencia), estado (ACTIVA INACTIVA), creado_en
Producto	id, marca_id, nombre, descripcion, estado (ACTIVO INACTIVO), metodologia_rotacion (FIFO LIFO), creado_en, actualizado_en
ProductoCategoria	producto_id, categoria_id ← PK compuesta (N-N)
Variante	id, producto_id, codigo (único), nombre, descripcion, precio_minorista_centavos, precio_mayorista_centavos, estado (ACTIVA INACTIVA), creado_en, actualizado_en
ImagenVariante	id, variante_id, referencia_imagen, es_principal, orden

Esto da las relaciones que el núcleo obligatorio exige:

- **1-N:** Marca → Producto y Producto → Variante.
- **N-N:** Producto ↔ Categoria vía producto_categoria.

metodologia_rotacion es del producto y se aplica a todas sus variantes. Lo usa Agustín para asignar lotes.

Los estados son listas cerradas, así que van como **enum de Postgres** o **varchar con un CHECK**, no como **varchar** suelto. Con un **varchar** libre un día entra "activa" en minúscula y rompe todas las consultas.

1.2 Migración — migrations/schema/catalogo.go

Agustín dejó un stub con `func MigracionesCatalogo() []*gormigrate.Migration` devolviendo vacío. Reemplazá su contenido. **Ese archivo es tuyo a partir de ahora.**

Una sola migración, ID `20261004_02_catalogo`. Corre segunda, después de identidad.

Obligatorio:

- Único en `marca(nombre)` y en `variante(codigo)`.

- Índices en `producto(marca_id)`, `variante(producto_id)`, `imagen_variante(variante_id)`, `producto_categoria(categoria_id)`, `categoria(padre_id)`. Son las FKs por las que se filtra en cada pantalla del catálogo: sin índice, cada listado recorre la tabla entera.
- Índice para la búsqueda por nombre: `CREATE INDEX ... ON variante (lower(nombre))`, porque la búsqueda no distingue mayúsculas de minúsculas y sin eso el índice no se usa.
- `onDelete` **explícito en cada FK, con el motivo en un comentario**:
 - `variante → producto`: `RESTRICT`. Un producto con variantes vendidas no se borra, se desactiva.
 - `producto → marca`: `RESTRICT`.
 - `imagen_variante → variante`: `CASCADE` (una imagen sin su variante no significa nada).
 - `producto_categoria`: `CASCADE` en los dos lados (es una tabla de unión).

La pregunta para cada relación es siempre la misma: *si borro esto, ¿qué otra cosa tiene sentido que desaparezca?* Casi siempre la respuesta es "ninguna", y por eso casi todo es `RESTRICT`. Los asistentes ponen `CASCADE` por costumbre: revisá el SQL generado antes de commitear.

```
git add backend/internal/core/domain/catalogo.go \
        backend/internal/adapters/db/postgresql/migrations/schema/catalogo.go
git commit -m "feat(catalogo): entidades de marca, categoria, producto, variante e imagen con su migracion"
git push
```

1.3 Repositorios — `repositories/catalogo/`

- `productos.go`: `Crear`, `PorID`, `Actualizar`, `CambiarEstado`, `Listar(filtros, limit, offset)`, `ReemplazarCategorias(productoID, categoriaIDs)`.
- `variantes.go`: `Crear`, `PorID`, `PorCodigo`, `Actualizar`, `CambiarEstado`, `ListarPublicas(filtros, orden, limit, offset) ([]Variante, int64, error)` — devuelve también el total para la paginación, con un `Count` aparte.
- `categorias.go`, `marcas.go`, `imagenes.go`: CRUD simple y `DefinirPrincipal(varianteID, imagenID)` en una transacción (poner una en `true` implica poner las otras en `false`).

Dos cosas que deciden la nota en este bloque:

- **Select explícito, nunca la fila entera.** El listado público no devuelve `costo`, ni datos del proveedor, ni nada que el comprador no tenga por qué ver. Un `Preload` sin `Select` que arrastra el producto entero con su marca y sus categorías es la fuga de datos más común que existe, y no se nota porque la pantalla queda igual.
- **Nada de N+1.** Para traer las variantes con su imagen principal y su marca, usá un `Preload` con `Select` o un `Join`, nunca un `for` que consulta por cada variante. Con 20 variantes anda instantáneo; con 2.000 son 2.001 viajes a la base y 4 segundos de pantalla en blanco. Se pregunta en la defensa.

```
git add backend/internal/adapters/db/postgresql/repositories/catalogo/
git commit -m "feat(catalogo): repositorios con select explícito, limite y sin consultas N+1"
git push
```

BLOQUE 2 — Catálogo público

SPEC-H04 y SPEC-H05 . Estos endpoints **no llevan sesión, a propósito**: la tienda se navega sin cuenta. Dejá un comentario arriba de cada uno diciéndolo, porque la diferencia entre "es abierto a propósito" y "se nos olvidó protegerlo" es exactamente lo que se pregunta.

Método y ruta	Qué hace	Errores
GET /api/catalogo/variantes	Listado con búsqueda, filtros, orden y paginación	400
GET /api/catalogo/variantes/{id}	Detalle de una variante, con sus imágenes y las otras variantes activas del mismo producto	404
GET /api/catalogo/categorias	Árbol de categorías activas	—
GET /api/catalogo/marcas	Marcas activas	—

Parámetros de GET /api/catalogo/variantes , **todos validados** (vienen de la query, que es tan pública como el body):

```
q=<texto>           búsqueda por nombre, sin distinguir mayúsculas
categoria=<uuid>     incluye las subcategorías de esa categoría
marca=<uuid>
orden=precio_asc|precio_desc|nombre_asc    (default: nombre_asc)
pagina=<n>           default 1, mínimo 1
porPagina=<n>       default 24, máximo 100
```

Criterios de aceptación de la spec que hay que cumplir:

- Búsqueda, filtros y orden **se combinan**. La paginación conserva la selección.
- No se repiten variantes entre páginas**: el orden tiene que ser estable, así que desempata siempre por `id` como último criterio. Sin eso, dos páginas pueden traer la misma fila.
- Las variantes **activas sin stock siguen visibles**, marcadas `SIN STOCK` . Las **inactivas no se muestran nunca**, tengan o no existencias: desactivar no es lo mismo que agotarse.
- Si no hay resultados, 200 con la lista vacía y el total en 0. **No un 404**: la búsqueda funcionó, simplemente no encontró nada.

Respuesta:

```
{
  "items": [ { "id": "...", "codigo": "...", "nombre": "...", "marca": "...",
               "precioMinoristaCentavos": 1590000, "imagenPrincipal": "...",
               "disponible": true } ],
  "pagina": 1, "porPagina": 24, "total": 137
}
```

El campo `disponible` sale del servicio de stock de Agustín (`inventario.NuevoServicioStock(db).Disponible(...)`). **Pedilo en lote, no uno por variante**: si llamás adentro del `for` que arma la respuesta, acabás de escribir un `N+1`. Si el servicio todavía no está en `develop` , devolvé `disponible: true` fijo con un `TODO` y conectalo después.

```
git add backend/internal/adapters/rest/handlers/catalogo/ \
      backend/internal/adapters/rest/dtos/catalogo/ \
      backend/internal/usecases/catalogo/
git commit -m "feat(catalogo): listado publico con busqueda, filtros, orden estable y paginacion"
git push
```

BLOQUE 3 — El CRUD completo de Producto (es el del núcleo obligatorio)

SPEC-H08 . Todo bajo `mw.RequierePermiso("catalogo.escribir")` .

Este es el **CRUD completo sobre al menos una entidad** que exige el núcleo obligatorio, así que tiene que estar entero y con los status correctos. Es lo primero que se mira en el criterio de API.

Método y ruta	Qué hace	Status OK	Errores
POST <code>/api/admin/productos</code>	Crea producto con marca, categorías y metodología	201 + el producto creado con su id	400, 401, 403, 409
GET <code>/api/admin/productos</code>	Listado paginado, incluye inactivos	200	401, 403
GET <code>/api/admin/productos/{id}</code>	Detalle con variantes y categorías	200	401, 403, 404
PATCH <code>/api/admin/productos/{id}</code>	Edita los campos permitidos	200	400, 401, 403, 404
DELETE <code>/api/admin/productos/{id}</code>	Desactiva , no borra	204 sin cuerpo	401, 403, 404, 409

Lo mismo para variantes (`/api/admin/variantes`), más:

Método y ruta	Qué hace
POST <code>/api/admin/variantes/{id}/imagenes</code>	Agrega una imagen
DELETE <code>/api/admin/imagenes/{id}</code>	Quita una imagen
PUT <code>/api/admin/variantes/{id}/imagen-principal</code>	Define cuál es la principal

Y el ABM de categorías y marcas (`POST` , `GET` , `PATCH` , `DELETE` lógico).

Los detalles que se corrigen en el code review:

- PATCH** , no **PUT** , para editar: con `PUT` el cliente tiene que mandar el objeto completo y un campo que falte se borra. El DTO de `PATCH` tiene todos los campos como punteros o `omitempty` , y sólo se copia lo que vino.
- DELETE** devuelve **204 sin cuerpo**. La respuesta viene vacía a propósito: no es un error.
- DELETE** desactiva, no borra. Las compras históricas tienen que mantener la referencia al producto. Si el producto ya tiene variantes con lotes o pedidos, borrarlo de verdad rompería el historial: por eso es desactivación lógica y por eso las FKs son `RESTRICT` .
- Un `codigo` de variante repetido es **409**, no 400: el request está perfecto, es el estado del sistema el que no lo permite. La prueba práctica: *¿el cliente puede arreglarlo cambiando lo que manda?* Si sí es 400; si tiene que pasar otra cosa en el sistema, es 409.

- **El stock no se toca desde acá.** Editar un producto nunca cambia existencias: eso va por lotes y ajustes, que son de Agustín. Lo dice la spec explícitamente.

```
git add backend/internal/adapters/rest/handlers/catalogo/ \
    backend/internal/adapters/rest/dtos/catalogo/
git commit -m "feat(catalogo): CRUD completo de producto y variante con desactivacion logica"
git push
```

BLOQUE 4 — El lector para los otros módulos

`backend/internal/usecases/catalogo/lector.go` . Esta firma está congelada: Nicolás ya está programando el checkout contra ella. No la cambies sin avisarle.

```
type VarianteVendible struct {
    ID                uuid.UUID
    ProductoID        uuid.UUID
    Codigo            string
    Nombre            string
    PrecioMinoristaCentavos int64
    Activa            bool
}

func NuevoLector(db *gorm.DB) *Lector
func (l *Lector) VariantePorID(ctx context.Context, id uuid.UUID) (VarianteVendible, error)
```

Devuelve `apierr.ErrNoEncontrado` si la variante no existe o está inactiva. Para el checkout, una variante desactivada no existe.

Esto es lo que hace que el precio del pedido no salga nunca del body: el checkout de Nicolás pide el precio acá, del servidor, y jamás confía en el que manda el navegador. Si el precio viniera del body, cualquiera compraría a \$1 desde Postman.

```
git add backend/internal/usecases/catalogo/lector.go
git commit -m "feat(catalogo): lector de variantes vendibles para checkout e inventario"
git push
```

Avisale a Nicolás apenas esto esté en `develoP` .

BLOQUE 5 — Datos, pruebas y cierre

5.1 Seed del catálogo

`backend/cmd/seed-catalogo/main.go` , idempotente (con `FirstOrCreate` , para poder correrlo muchas veces sin duplicar):

- 3 marcas reales del negocio (Victoria's Secret y las que correspondan).

- 4 categorías con al menos una subcategoría: Perfumes, Cremas, Body splash, Electrónicos.
- 8 productos, cada uno con 1 a 3 variantes, con precios reales en centavos.
- Al menos **un producto inactivo** y **una variante inactiva**, para poder mostrar en la demo que no aparecen en el catálogo público.

Coordiná los ids de las variantes con Agustín: él siembra los lotes encima de ellas y necesita que una quede con stock, una con una sola unidad y una con cero.

5.2 docs/api/catalogo.http

@base en `http://localhost:8080` con la línea de producción comentada al lado. Por cada endpoint, **tres requests**: la que anda, la del body inválido (400, no 500) y la del recurso que no existe (404). Arriba de cada una, el status esperado escrito.

Y estas, que son las que muestran que el módulo cumple la spec:

```
### buscar con filtro y orden combinados → 200, orden estable
GET {{base}}/api/catalogo/variantes?q=love&categoria={{idCremas}}&orden=precio_asc&pagina=1

### una búsqueda sin resultados → 200 con lista vacía, NO 404
GET {{base}}/api/catalogo/variantes?q=zzzzzz

### crear producto sin token → 401
POST {{base}}/api/admin/productos

### crear producto con token de CLIENTE → 403
POST {{base}}/api/admin/productos
Authorization: Bearer {{tokenCliente}}

### crear variante con un código ya usado → 409
POST {{base}}/api/admin/variantes
Authorization: Bearer {{tokenDueno}}
```

Mirá el status, no el cuerpo: un 500 donde iba un 409 se ve igual de roto desde afuera y sólo el número los distingue.

Después del merge, descomentá la @base de producción y corré el archivo entero otra vez. **Un endpoint que anda en localhost y no en la URL pública, para esta materia, no existe.**

```
git add backend/cmd/seed-catalogo/ docs/api/catalogo.http
git commit -m "feat(catalogo): seed con datos reales y requests de prueba de los tres casos"
git push
```

5.3 Lo que te toca como responsable del repositorio

Esto es tan importante como tu módulo, porque sin esto no hay entrega:

1. Revisá y mergeá los PR de los otros tres a `develop`, usando el checklist de la plantilla. **Nadie aprueba su propio PR**: es regla de la cátedra y se mira en la nota de concepto.
2. Cuando `develop` esté estable y desplegado, abrí el PR `develop → main` y mergealo. El CI de Agustín despliega con el push a `main`.

3. Verificá que los cuatro tengan commits repartidos a su nombre. Un integrante sin contribución verificable en el repositorio **desaprueba la instancia aunque el equipo saque 10**.

```
gh pr create --base develop --title "Catalogo" --body "Marcas, categorías, productos y variantes con relación N-N, catálogo público con búsqueda, filtros y paginación estable, y CRUD completo de producto."
```

Definición de terminado

No te detengas hasta que todo esto sea cierto:

- ☐ `go build ./... && go vet ./... && go test ./...` en verde.
- ☐ La migración de catálogo aplicada en la base de la nube.
- ☐ `GET /api/catalogo/variantes?q=...&categoria=...&orden=precio_asc` funciona con los tres
- ☐ Una búsqueda sin resultados devuelve 200 con lista vacía.
- ☐ Una variante inactiva **no** aparece en el catálogo público.
- ☐ Una variante activa sin stock **sí** aparece, marcada como no disponible.
- ☐ El CRUD de producto está completo: 201 al crear, 200 al leer y editar, 204 al desactivar.
- ☐ Sin token, `POST /api/admin/productos` da 401. Con token de cliente, 403.
- ☐ `docs/api/catalogo.http` corre entero **contra producción**.
- ☐ `main` protegida, CODEOWNERS y plantilla de PR en el repo.
- ☐ Todo pusheado, con commits repartidos y a tu nombre.

Si te trabás

- Si el kernel de Agustín todavía no está en `develop`, seguí con el dominio, la migración y los repositorios: no dependen de él.
- Si el middleware de Yasmín todavía no está, escribí los endpoints públicos primero y dejá los de administración con el mount del middleware como último paso.
- Si una migración falla, **nunca** edites una ya aplicada ni corras un reset contra la base de la nube. Se corrige con una migración nueva.
- Si algo del alcance no llega, es preferible un endpoint menos que uno que devuelve 500.

Lo que te van a preguntar el martes

1. **Mostrame qué pasa si le pido esto con la cuenta de otro.**
2. ¿Por qué los precios están en `int64` de centavos y no en `float`?
3. ¿Por qué `Producto` y `Variante` son dos tablas y no una?
4. ¿Dónde está la relación N-N y por qué esa y no otra?
5. ¿Qué pasa si borro una marca que tiene productos? ¿Por qué `RESTRICT` y no `CASCADE`?
6. Mostrame el listado con 2.000 variantes: ¿cuántas consultas hace? (la pregunta del N+1)
7. ¿Por qué `DELETE` desactiva en vez de borrar?
8. ¿Por qué el catálogo es público y `/api/admin/productos` no? ¿Dónde se decide eso?